

PONTIFICIA UNIVERSIDAD CATÓLICA MADRE Y MAESTRA



ASIGNATURA:

ASEGURAMIENTO DE CALIDAD DE SOFTWARE

GRUPO:

1890-4346

PRESENTADO POR:

BRIGIBEL YSAURA MARTÍNEZ MARTÍNEZ, 1014-4327

STARLIN JOSÉ FRIAS LUNA, 1014-3611

TEMA:

GESTIÓN DE RIESGOS

PRESENTADO A:

ING. FREDDY A. PEÑA

FECHA DE ENTREGA:

JUEVES 3 DE JULIO DEL 2025

SANTIAGO, REPÚBLICA DOMINICANA

Gestión de Riesgos

En el desarrollo del Sistema de Gestión de Inventarios con QAS, se identificaron varios riesgos que podrían afectar el éxito del proyecto. A continuación, se describen estos riesgos, su impacto potencial y las estrategias detalladas para su mitigación:

1. Riesgo: Incompatibilidad entre versiones de dependencias

Descripción:

Durante el desarrollo, es posible que se presenten conflictos de versiones entre las librerías utilizadas en el frontend (React, Playwright) y el backend (Spring Boot, dependencias de seguridad). Esto podría generar errores de compilación, incompatibilidades en tiempo de ejecución o vulnerabilidades de seguridad.

Impacto:

Podría impedir el despliegue estable o generar fallos inesperados en producción.

Estrategia de mitigación:

- Fijar versiones estables de todas las dependencias en los archivos package.json y build.gradle.
- Configurar un archivo package-lock.json o yarn.lock en el frontend para mantener la coherencia en cada instalación.
- Probar las actualizaciones de dependencias en entornos de desarrollo separados antes de aplicarlas en la rama principal.
- Documentar las versiones utilizadas para futuras referencias.
- Utilizar las versiones estables de las librerías y estándar.

2. Riesgo: Fallos de seguridad en la autenticación JWT

Descripción:

El sistema utiliza tokens JWT para manejar la autenticación. Por lo que, un error en la implementación podría permitir accesos no autorizados, suplantación de usuarios o exponer datos sensibles.

Impacto:

Un fallo de seguridad puede comprometer la integridad y confidencialidad del sistema, afectando la confianza del cliente.

Estrategia de mitigación:

- Implementar pruebas automatizadas de login y validación de token, verificando casos como tokens expirados o manipulados.
- Configurar tiempos de expiración adecuados para los JWT y mecanismos de renovación segura (refresh tokens si fuera necesario).
- Usar HTTPS para proteger la transmisión de tokens.
- Revisar la implementación conforme a las mejores prácticas de seguridad OWASP.
- Capacidad de invalidar tokens de forma manual como administrador para ciertos usuarios.

3. Riesgo: Error en la integración frontend-backend**Descripción:**

El frontend y el backend deben comunicarse a través de una API REST. Cambios en los endpoints, datos enviados o estructuras de respuesta pueden provocar errores en la integración y afectar la experiencia del usuario.

Impacto:

Podría causar mal funcionamiento en las operaciones CRUD, bloqueando la gestión de productos.

Estrategia de mitigación:

- Documentar los endpoints de la API y mantener consistencia en las rutas.
- Implementar pruebas integrales con Playwright para simular la interacción completa desde el navegador hasta el backend.
- Usar controladores de error en el frontend para manejar de forma clara los posibles errores devueltos por la API (por ejemplo, 401, 404, 500).
- Realizar uso de los DTO para directamente no interactuar con la entidad de la base de datos y evitar errores de integridad.

4. Riesgo: Pérdida de avance por errores en Git

Descripción:

El uso incorrecto de Git puede ocasionar pérdida de commits, sobrescritura de código o conflictos difíciles de resolver, sobre todo cuando se trabaja en equipo.

Impacto:

La pérdida de trabajo representa retrasos y tener que volver a trabajar innecesariamente.

Estrategia de mitigación:

- Definir un flujo de trabajo basado en ramas (main, develop, feature/xxx).
- Hacer commits pequeños y frecuentes con mensajes claros.
- Actualizar las ramas frecuentemente antes de trabajar con algún feature para evitar que no hayan cambios pendientes con los cuales no contamos localmente y así evitar conflictos.
- Realizar push regular a un repositorio remoto (GitHub) para tener una copia de seguridad.
- Utilizar pull requests para revisión de código y evitar conflictos en la rama principal.

5. Riesgo: Retraso en el cronograma por falta de experiencia en herramientas

Descripción:

El desconocimiento parcial o total de herramientas como Spring Boot, React, Playwright, Docker, entre otras, podría ralentizar el desarrollo o llevar a implementaciones incorrectas.

Impacto:

Podría impedir completar el proyecto dentro del plazo establecido.

Estrategia de mitigación:

- Invertir tiempo en investigar las tecnologías durante la primera semana del proyecto, incluyendo guías, tutoriales y pruebas de concepto.
- Dividir las tareas según las fortalezas y experiencia previa de cada miembro del equipo para maximizar la productividad.
- Comunicación constante entre los miembros del equipo con respecto a dudas técnicas.
- Documentar dudas y soluciones para compartir el conocimiento entre los integrantes del equipo.